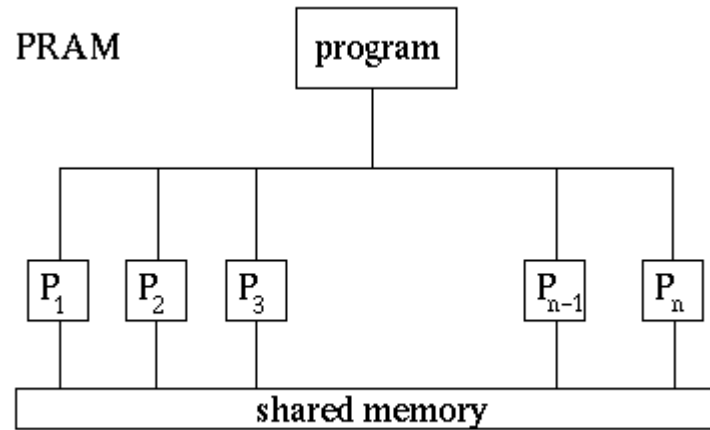




Parallel Computing - Exercises

Session 1:

- PRAM
- Performance Metrics
- Divide and Conquer
- Scaling Laws

Data Parallelism

Overall the best strategy for **scalable parallelism**.

Def:

Any kind of parallelism that grows as the data set grows or, more generally, as the problem size grows.

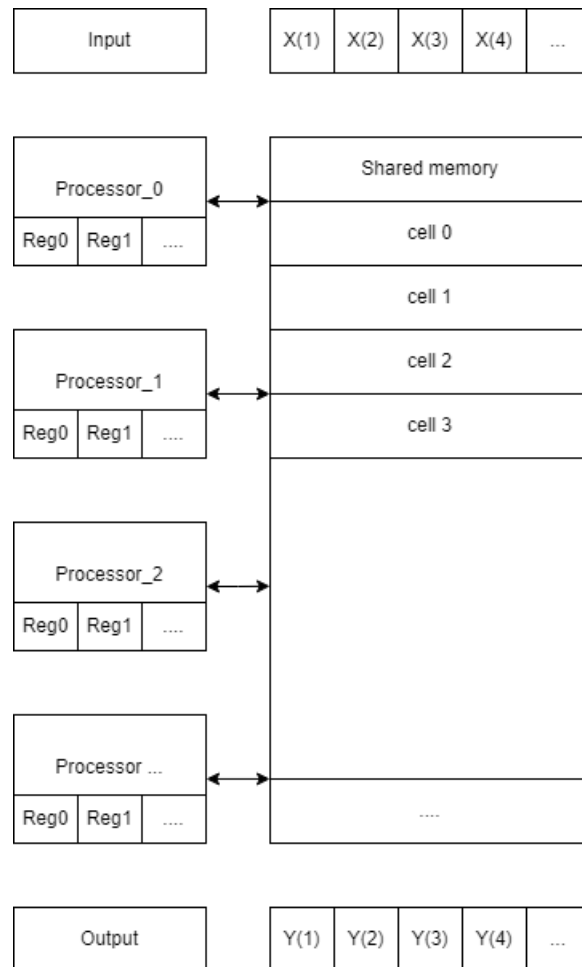
Typically the data is split into chunks and each chunk processed with a separate task. Bigger data sets generate more tasks.

Splitting may be **flat** (parallel for) or **recursive** (nested calls).

The **span** is the critical path or, equivalently, the longest chain of operations.

Alternative paradigm: [functional decomposition \(→ wikipedia\)](#)

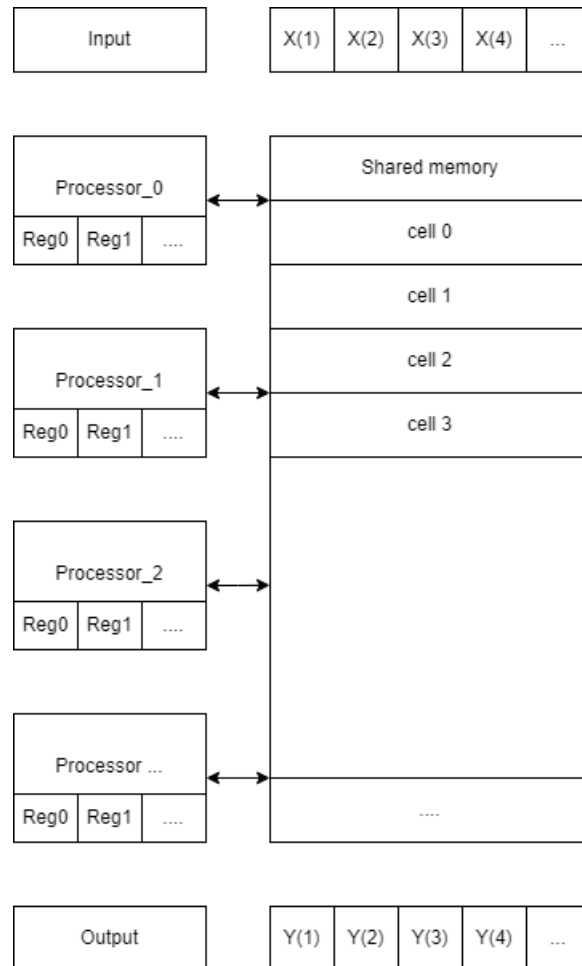
PRAM (parallel random access machine)



- $p \geq 1$ processors operating concurrently
 - For us: each processor runs the same program (abstracts both SIMD and SPMD systems)
- share (unlimited) memory
 - n : data set size
 - $X[n]$: input vector
 - $Y[n]$: output vector
 - $A[n]$, $B[n]$, ... : support vectors

This isn't a mandatory structure: if needed, other input or output structures can be defined.
Like another vector $Z[n]$, a matrix $M[n \times m]$, etc.

PRAM (parallel random access machine)



- “Mental sanity” assumptions
 - Numbers have unlimited precision
 - Symmetric processors (all identical)
 - All instructions take 1 cycle
 - All memory accesses take 1 cycle
 - Registers (aka named local variables) are private to each processor
 - Each processor knows its own index i
- Computation steps of a processor:
 - Read once from X and read once from one of A , B , ...
 - Unlimited accesses to local variables
 - Perform one computation
 - Write once to Y

Syntax Overview

Simple example: sum consecutive pairs of numbers.

HP: $p \geq n$

```
FUNCTION sum(X, Y, n)
```

```
  IF  $i + 1 < n$ :
```

```
     $a = X[i]$ 
```

```
     $b = X[i + 1]$ 
```

```
     $Y[i] = a + b$ 
```

Very easy situation:

- No data exchanged between processors
- More processors than tasks

Complexity: $O(2)$ (constant time)

Concurrent/Exclusive Reads/Writes

The PRAM's memory model is characterized by how it allows multiple processors to access the same memory location:

- **ER**: at most 1 processor can read a location at once
- **CR**: > 2 processors can read the same location at once

ER can be useful to carry out atomic operations, CR is more common.

- **EW**: at most 1 processor can write the same location at once
- **CW**: > 2 processors can write the same location at once

CW requires arbitration for who ends up writing.

Example on colab of CW: disjunctive normal form (DNF) with pthreads.

For today, assume a CREW machine!

Performance Metrics

$T^*(n)$: Time to solve problem of input size n on one processor, using **best sequential algorithm**.

$T_p(n)$: Time to solve problem of input size n on p processors, using a given data parallel algorithm.

$$\Rightarrow T_1(n) \geq T^*(n) \geq T_{p>1}(n) \geq T_\infty(n)$$

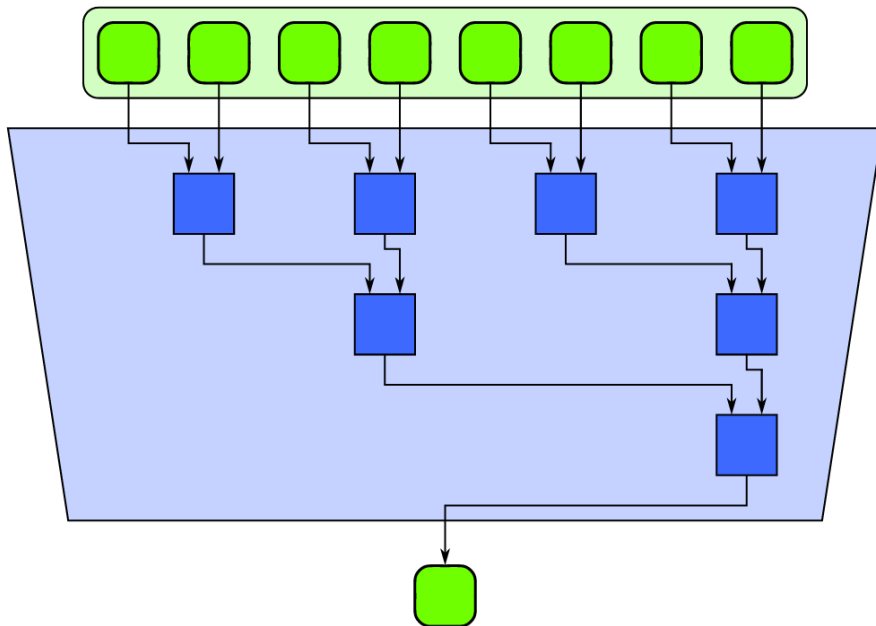
$SU_p(n) = \frac{T^*(n)}{T_p(n)}$: Speedup on p processors.

$E_p(n) = \frac{T_1(n)}{p \cdot T_p(n)}$: Efficiency on p processors.

$W(n)$: Work, total number of unit-time operations.

Array Σ ($p \geq n$)

Add together all n elements of an array.



HP: $p \geq n$

FUNCTION array_sum(X , Y , n)

IF $n = 1$ and $i = 0$:

$Y[0] = X[0]$ // result in $Y[0]$

return

// Processor i works on

// pair $(2i, 2i + 1)$

IF $2*i + 1 < n$:

$X[i] = X[2*i] + X[2*i + 1]$

ELSE IF $2*i < n$: // odd length

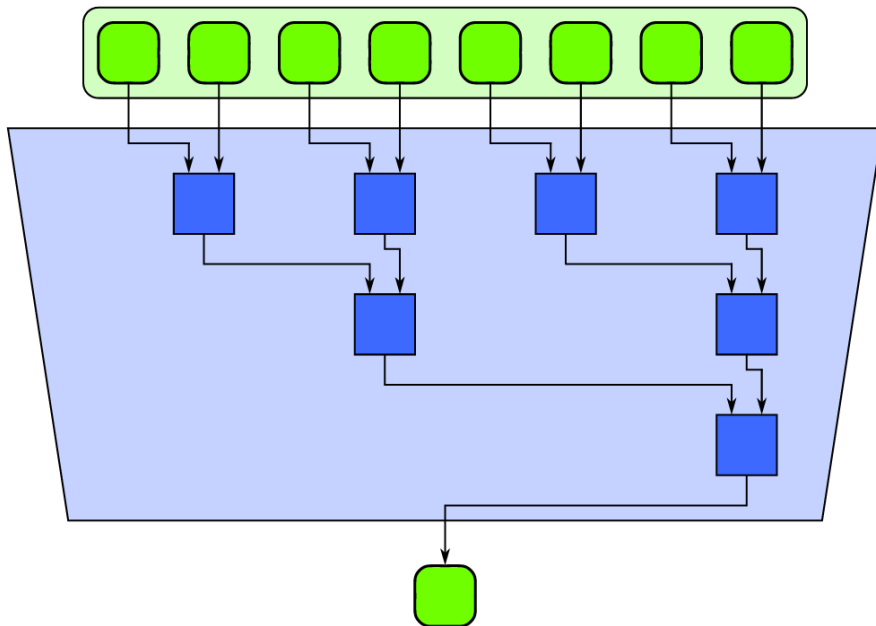
$X[i] = X[2*i]$

array_sum($X[0:n/2]$, Y , $\text{ceil}(n/2)$)

Core operation
goes here!

Array Π ($p \geq n$)

Multiply together all n elements of an array.



HP: $p \geq n$

```
FUNCTION array_prod(X, Y, n)
```

```
    IF  $n = 1$  and  $i = 0$ :
```

```
         $Y[0] = X[0]$  // result in  $Y[0]$ 
```

```
    return
```

```
    // Processor  $i$  works on
```

```
    // pair  $(2i, 2i + 1)$ 
```

```
    IF  $2*i + 1 < n$ :
```

```
         $X[i] = X[2*i] * X[2*i + 1]$ 
```

```
    ELSE IF  $2*i < n$ : // odd length
```

```
         $X[i] = X[2*i]$ 
```

```
    array_prod( $X[0:n/2]$ , Y,  $\text{ceil}(n/2)$ )
```

Core operation
goes here!

Array Σ or Π ($p \geq n$)

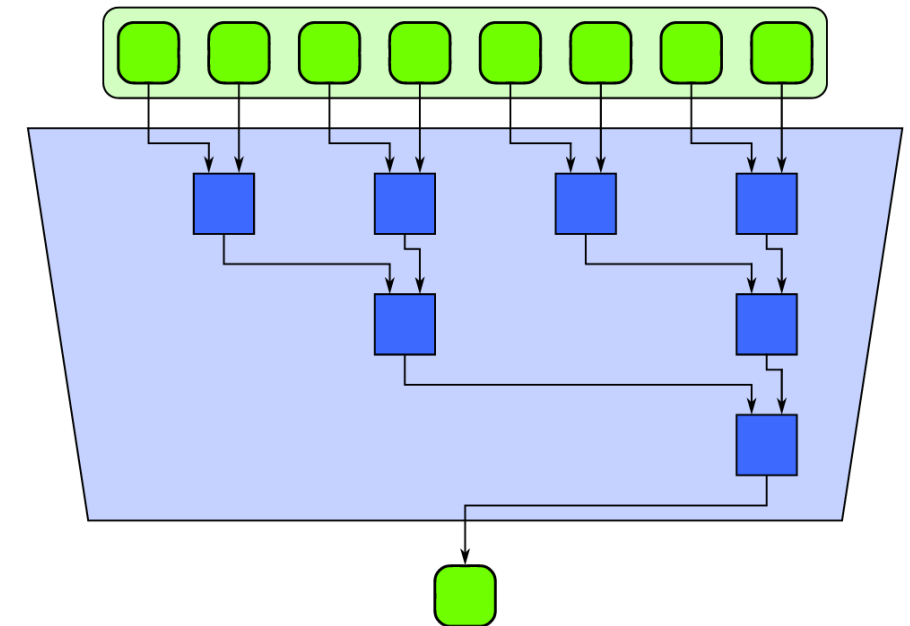
Complexity:

$$T_p(n) = \log(n)$$

Reason: the recursive calls create a binary tree of pair-wise operations.

$$T_1(n) = \sum_{i=1}^{\lceil \log n \rceil} \frac{n}{2^i} \approx n \quad T^*(n) = n$$

$$SU_p(n) = \frac{n}{\log(n)} \quad E_p(n) = \frac{1}{\log(n)} \quad W(n) = n$$



Fun fact: with floats, the parallel and sequential version may give different results, since their sum and product are not associative!

Array Σ or Π ($p < n$)

Removing the hypothesis $p \geq n$ complicates the algorithm quite a bit. Now processors chew through each tree level and some move to the next as soon as data dependencies are satisfied...

Complexity: $T_p(n) \approx \frac{n}{p} + \log(p)$

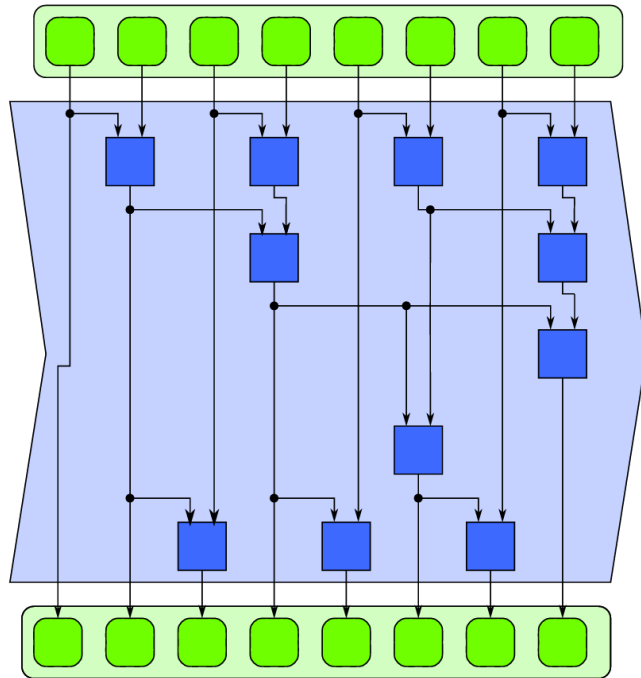
An intuition:

- First, each processor reduces its chunk of n/p inputs down to a single value.
- Then we still need to combine those p values. They are now far fewer and form a tree of height $\log(p)$.

Exact complexity? Outside the scope of this lecture...but if you are interested...

Prefix Sums ($p \geq n$)

Compute all prefix sums of an n -elements array.



HP: $p \geq n$

FUNCTION prefix_sums(X, Y, n)

IF $n == 1$ AND $i == 0$:

$Y[0] = X[0]$

return

// downsweep: like the array sum

IF $2*i + 1 < n$:

$X[i] = X[2*i] + X[2*i + 1]$

ELSE IF $2*i < n$:

$X[i] = X[2*i]$

prefix_sums($X, Y, \text{ceil}(n/2)$)

// upsweep: rebuild full prefix sums

IF $2*i + 1 < n$:

$Y[2*i] = Y[i] - X[i] + X[2*i]$

$Y[2*i+1] = Y[2*i] + X[2*i+1]$

ELSE IF $2*i < n$:

$Y[2*i] = Y[i] - X[i] + X[2*i]$

Prefix Sums ($p \geq n$)

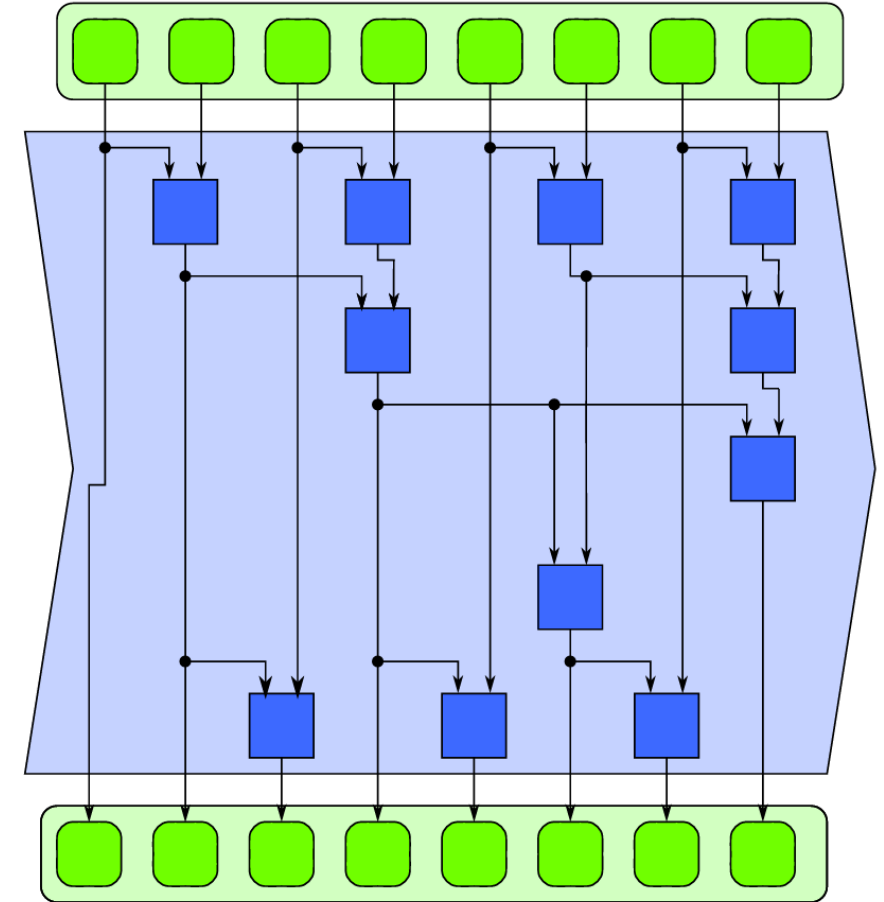
Complexity:

$$T_p(n) = \log(n)$$

Reason: there is one accumulation tree going down, and another going up, but we ignore the constant 2.

$$T_1(n) = \sum_{i=1}^{\lceil \log n \rceil} \left(\frac{n}{2^i} + \frac{n}{2^i} - 1 \right) \approx n$$

Reason: if there are k ops on a level of the downward tree, the prefix sum needs another $k - 1$ ops to finish the remaining values.



Prefix Sums ($p \geq n$)

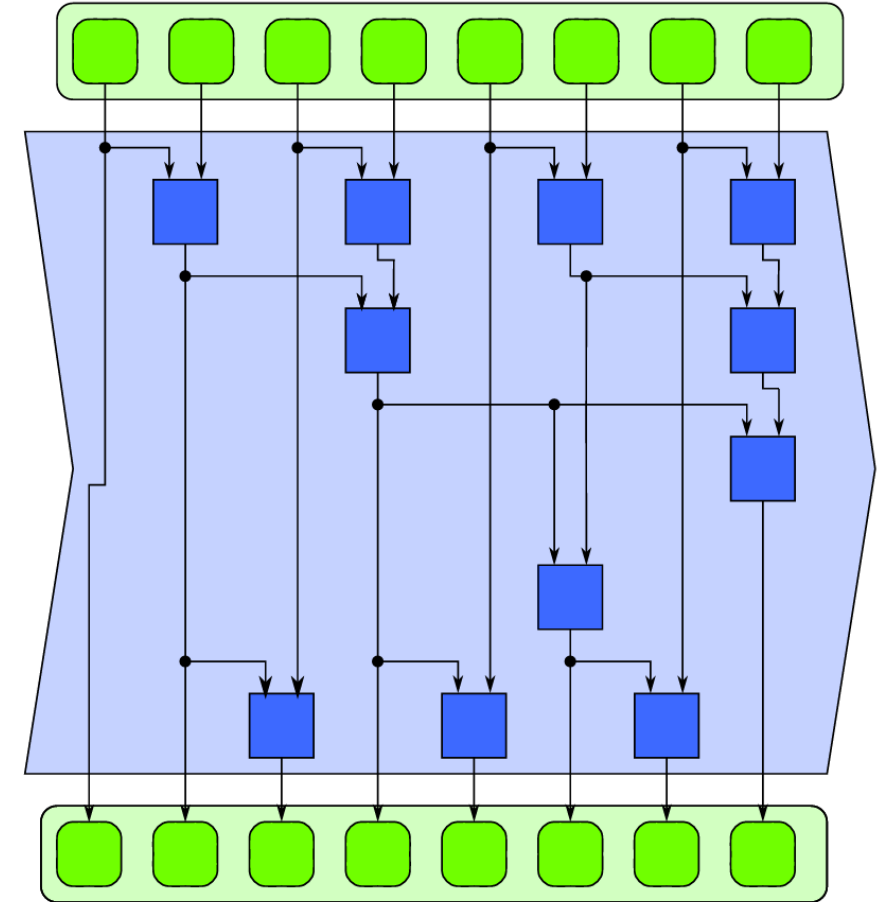
Continuing:

$$T^*(n) = n$$

$$SU_p = \frac{n}{\log(n)}$$

$$E_p(n) = \frac{1}{\log(n)}$$

$$W(n) = n$$



This can be micro-optimized quite a bit: [see this \(→ algorithmica\)!](#)

Performance Metrics – Interpreting Efficiency

$E_p(n) = \frac{T_1(n)}{p \cdot T_p(n)}$ = how much **useful** work we make the available processors do.

Example: multiply two scalar values together.

There is work for only one processor here: $T_p(n) = T_1(n) = 1$

$\Rightarrow E_p(n) = \frac{1}{p}$ only one processor does actual work, others are idle.

Example: compute the sum of an n -elements vector.

Logarithmic sum plus missing operation: $T_{p=n}(n) = \log(n)$, $T_1(n) = n$

$\Rightarrow E_{p=n}(n) = \frac{1}{\log(n)}$ we have more and more inactive processors as we continue.

Divide and Conquer

Repeatedly **subdivide** a problem in identical sub-problems until a **base case** is reached. Solve the base cases, then **combine** them to solve the original problem.

Subdivision can be done both iteratively and through recursive calls.

Typical of sequential algorithms, easily parallelized by **assigning each subproblem to a different processor**.

Usually complexity written as: $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$.

Number of sub-problems. Recursive call on b -times smaller problem. Cost of recombining + base case.

Master Method

For recurrences of the form $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$.

With $a \geq 1, b \geq 1$ and $f(n)$ asymptotically positive.

- $T(n) = \theta(n^{\log_b a})$ if $f(n) = O(n^{\log_b a - \epsilon})$ for $\epsilon > 0$
 - $T(n) = \theta(n^{\log_b a} \ln^{k+1} n)$ if $f(n) = \theta(n^{\log_b a} \ln^k n)$ and $k \geq 0$
 - $T(n) = \theta(f(n))$ if $f(n) = \Omega(n^{\log_b a + \epsilon})$ for $\epsilon > 0$ and $f(n)$ satisfies the regularity condition $af(n/b) \leq cf(n)$ with some $c < 1$
- Don't forget this little guy!
- 

There is also a more general version: [Akra-Bazzi method \(→ wikipedia\)](#)

Matrix Multiplication

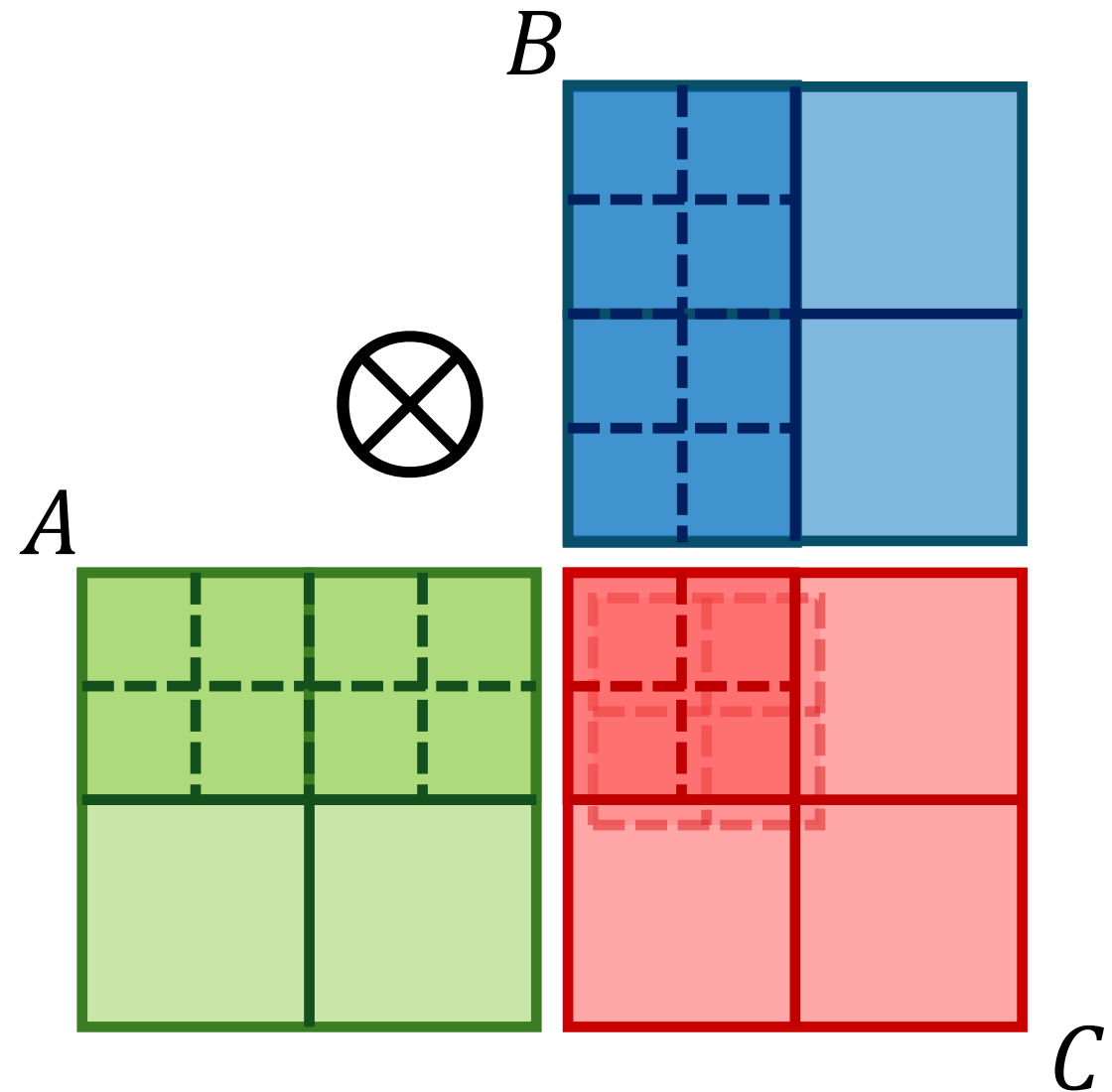
$$C = A \cdot B \quad A, B, C \in \mathbb{R}^{n \times n}$$

Repeatedly divide it into 8 matmuls until they all are 2×2 s. Then, each can be computed in constant time.

Assuming all square matrices. Let all their dimensions be n :

$$T(n) = 8 \cdot T\left(\frac{n}{2}\right) + \theta(n^2)$$

Master method: $T(n) = O(n^3)$ ☹



Base Case:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} x & y \\ z & w \end{bmatrix} = \begin{bmatrix} ax + bz & ay + bw \\ cx + dz & cy + dw \end{bmatrix}$$

Matrix Multiplication

Unfortunately, the n^2 contains:

- the constant 2×2 matmul
- the cost of combining back the 8 matrices in 4, with pair-wise element-wise sums, that is n^2

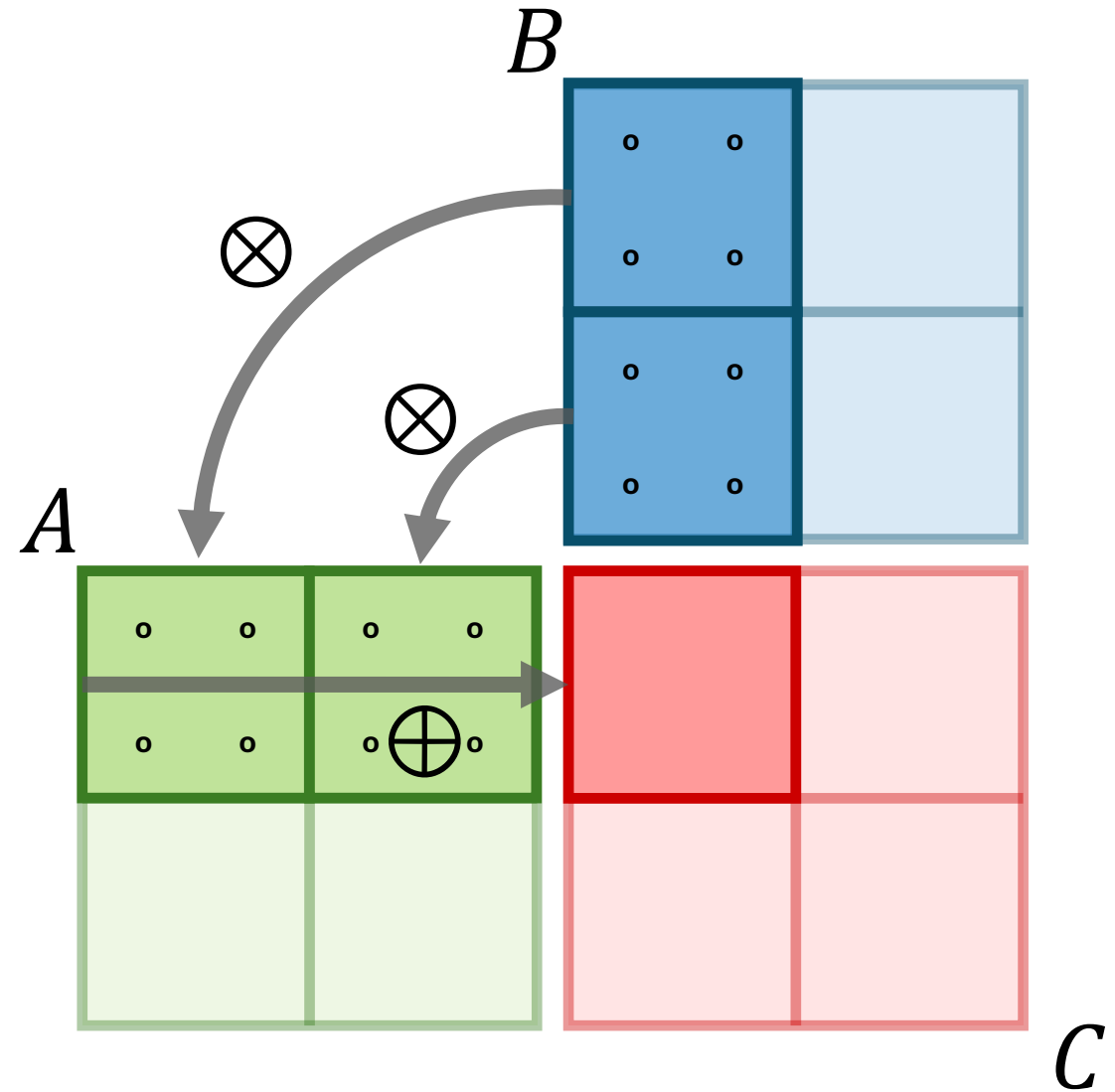
Divide and conquer doesn't help.

But we can go parallel!

Take $p \geq 8^{\log_2(n)-1} = (n/2)^3$ processors:

$$T(n) = T\left(\frac{n}{2}\right) + \theta(n^2)$$

Master method: $T(n) = O(n^2)$ ☺



Matrix Multiplication: Digression

There are “better” algorithms that improve on the idea:

- [Strassen \(→ Wikipedia\)](#) needs to compute only 7 out of 8 submatmuls by expressing the 8^{th} as a function of the others.
- [Winograd \(→ Wikipedia\)](#) pushes it further, reducing required matrix additions to 15 rather than 18 in Strassen.

But those are **cosmic algorithms**: their hidden constant in the complexity is large, not worth it for anything realistic.

While those can be parallelize, they shouldn't improve performance until matrices reach absurd sizes.

Merge Sort

HP: $p \geq n$

```
FUNCTION mergesort(X, lo, hi)
  IF lo+1 < hi: // >1 element
    mid = floor((lo + hi)/2)
    // hand off half the work
    // to another processor
    FORK mergesort(X, lo, mid)
    mergesort(X, mid, hi)
  JOIN
  merge(X, lo, mid, hi)
```

```
FUNCTION merge(X, lo, mid, hi)
  j, k = lo, mid
  WHILE j < mid or k < hi:
    IF (j < mid and
        (k == hi or X[j] <= X[k])):
      A[j + k] = X[j]
      j += 1
    ELSE
      A[i + j + k] = X[k]
      k += 1
  COPY A[0:hi - lo] -> X[lo:hi]
```

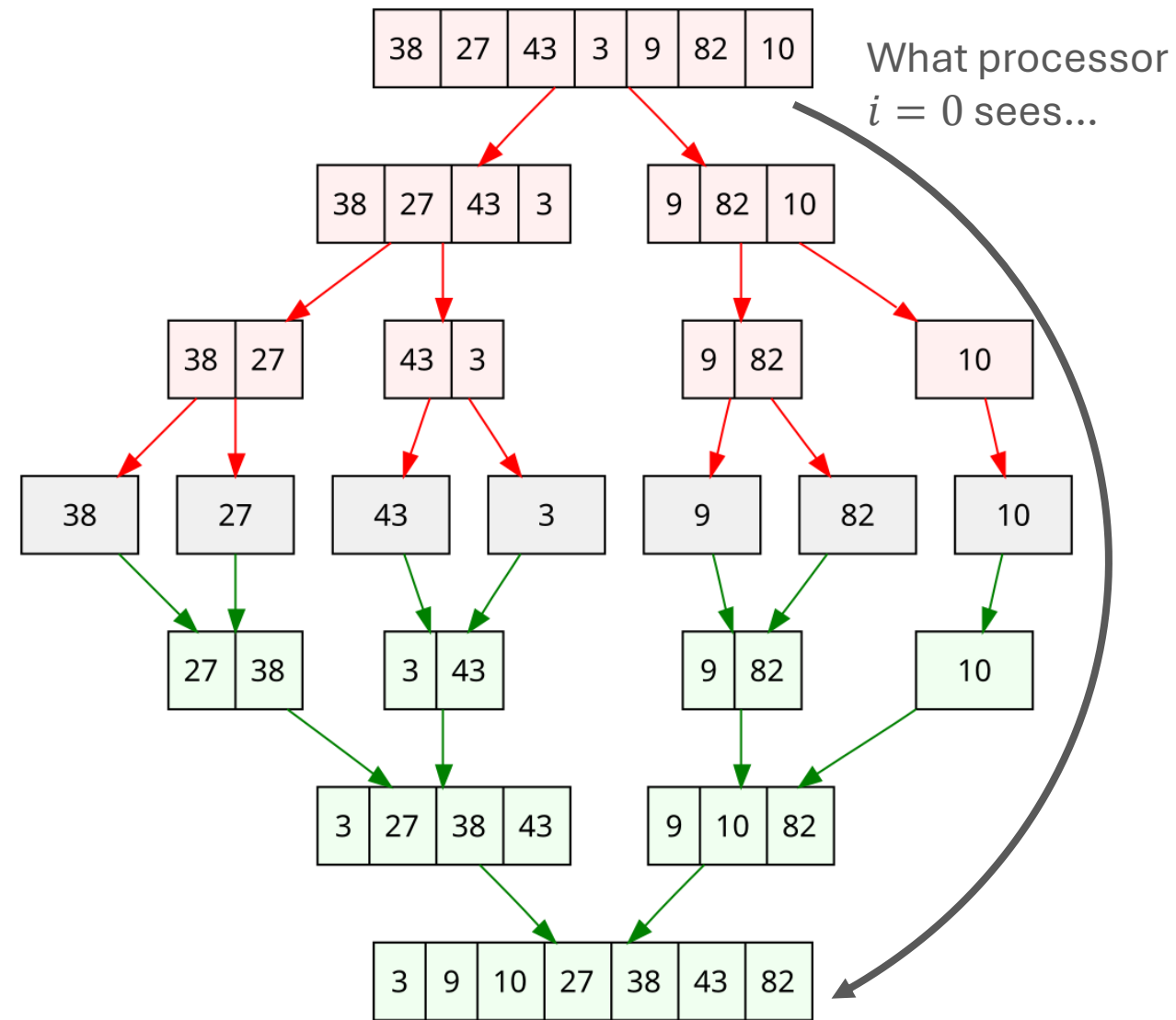
Merge Sort

Our parallel version gives to another processor one branch of each bifurcation.

This isn't necessarily the best parallel sort technique, but it gets the job done.

Parallel merge version:

- [Cormen's mergesort \(→ wikipedia\)](#)
- [Alternative page \(→ wikipedia\)](#)



Merge Sort

- Sequential (one processor): $T_1(n) = 2 \cdot T_1\left(\frac{n}{2}\right) + O(n)$
 - Have fun applying the Master Method 😊!

Cormer's version gets this down to $\log n$.

- Parallel version: $T_p(n) = T_p\left(\frac{n}{2}\right) + O(n)$, the cost of merging domaines as each processor sees once each level of the tree.
The resulting complexity is: $T_p(n) = n$

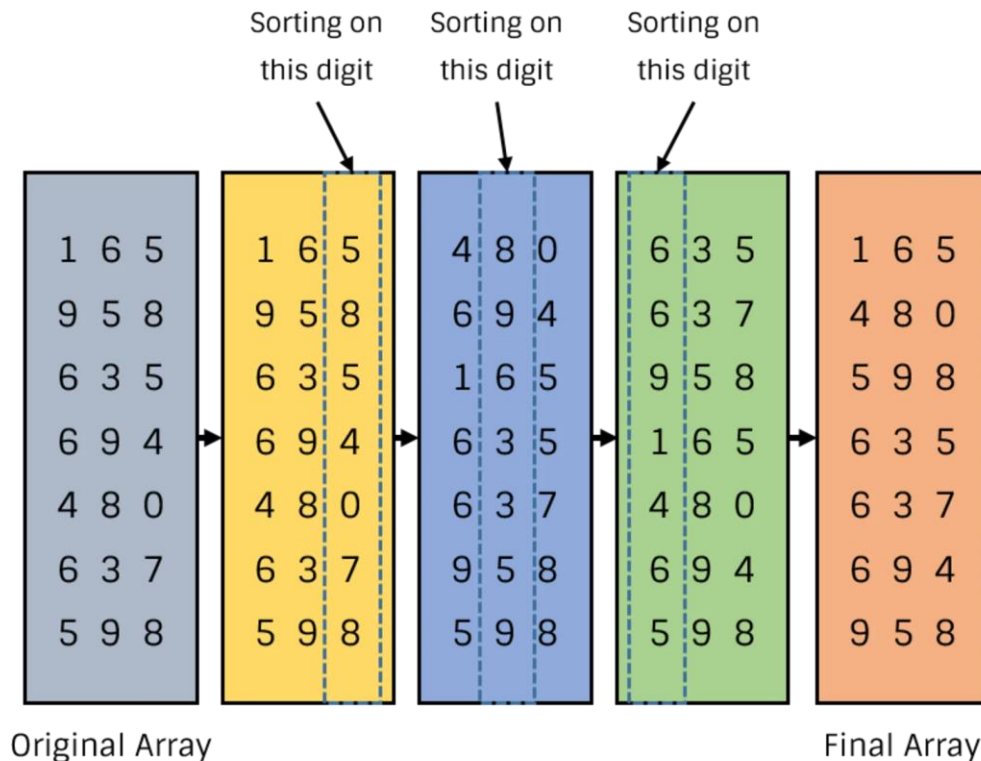
Since $a = 1, b = 2, \epsilon = 1$

$$\text{Speedup: } SU_p = \frac{n \cdot \log n}{n} = \log n$$

Note: you should know that the best sequential is $T^*(n) = n \cdot \log n$

Radix Sort

Optional opportunities
for extra parallelism...



```
// d = number of digits
// b = base of the representation
// processor i handles element i
FUNCTION radix_sort(X, n, b, d)
    FOR k = 0 TO d-1: // process each digit
        v = digit(X[i], k)
        PAR FOR u = 0 TO b-1: // build buckets
            A[u][i] = (v == u ? 1 : 0)
        PAR FOR u = 0 TO b-1: // prefix sums per bucket
            prefix_sums(A[u], B[u], n)
        IF i == 0: // compute bucket offsets
            C[0] = 0
            FOR u = 1 TO B - 1:
                C[u] = C[u - 1] + B[u - 1][n - 1]
        // reorder elements
        pos = 0[v] + P[v][i] - 1
        D[pos] = X[i]
    SYNC // wait for all processors
    COPY D -> X
```


Radix Sort

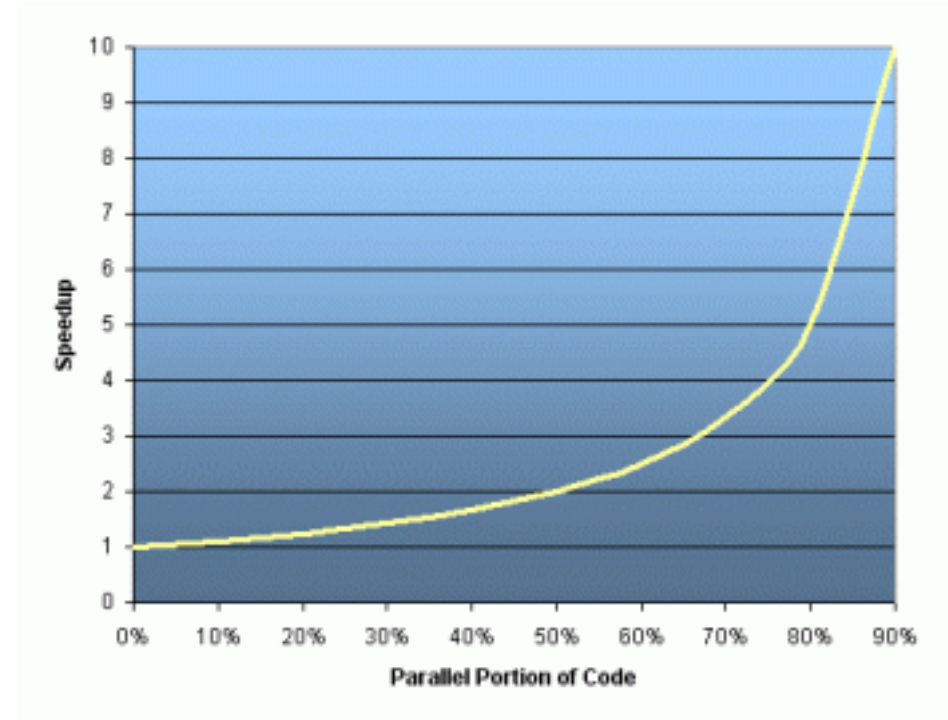
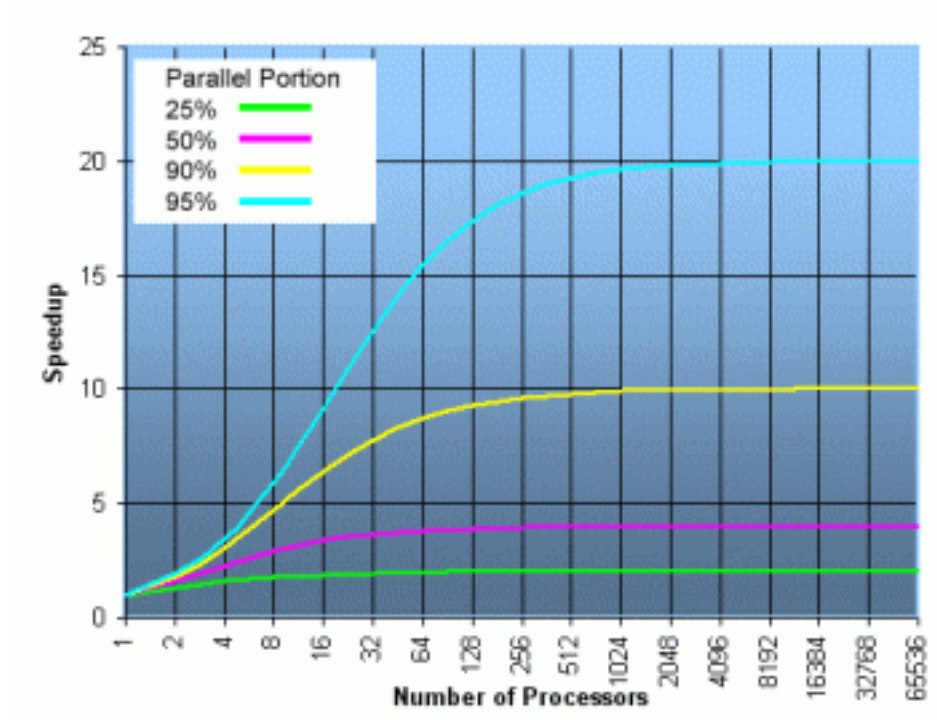
Assume $p \geq n$ for simplicity.

- Sequential (one processor): $T_1(n) = d \cdot (n + b)$
- Parallel version: $T_p(n) = d \cdot \log n$, assuming $O(\log b)$ dominated by $O(\log n)$ and with $O(\log n)$ coming from the prefix sums.

Speedup:

- $SU_p = \frac{n}{d}$ if $T^*(n) = n \cdot \log n$ using comparison-based sorting
- $SU_p = \frac{n}{d \cdot \log n}$ if $T^*(n) \approx n$ using non-comparative sorting

Scaling Laws: is there a limit to going // ?



More insight at [this great tutorial from the guys at the Lawrence Livermore National Laboratory!](#)

Image credit: [the same great tutorial!](#)

Sorry for the poor img. resolution ☹ ...

Scaling Laws: is there a limit to going // ?

Image: [Cray-1A \(→ wikipedia\)](#) supercomputer.



One way of doing things from 70s was to cram more functional units in the same giant CPU, going parallel with a vector processor, the SIMD forerunner of modern vector instructions.

The issue? It was still a single CPU, if your code had long **serial tasks**, or non-vectorizable code it could only do one thing at a time...

Yup, the Cray-1 was just one big CPU with 64-elements wide vector instructions. Its power consumption: 115 kW ...

Amdahl's Law

Math-wise, holds for $p \geq 1$
See it on [GeoGebra!](#)

Computational model: serial parts and parallel parts, **fixed problem size!**

Goal: run the same problem size faster by adding processors

$$T_p = \text{serial} + \frac{\text{parallel}}{p} \quad T_1 = \text{serial} + \text{parallel} \quad \Rightarrow \quad SU_p \leq \frac{T_1}{T_p} < p$$

The parallelizable part is always a fraction of the total work...

This is «=» if $T^* = T_1$

$$SU_{p=\infty} = \frac{1}{1 - \frac{\text{parallel}}{\text{serial} + \text{parallel}}} \left. \vphantom{\frac{1}{1 - \frac{\text{parallel}}{\text{serial} + \text{parallel}}}} \right\} \text{Fraction of parallel work.}$$

Here everything is normalized
w.r.t. n , thus $\text{serial} + \text{parallel} = 1$

Gustafson's Law

Here n does not appear:
this holds for any n !
Thus it grows as both n
and p grow!

Serial and parallel parts, but **fixed time** with increasing problem size!

⇒ s time fraction for serial work, $1 - s$ time fraction for parallel work

$$T_1 = s + p \cdot (1 - s) \quad T_p = s + (1 - s) = 1 \quad SU_p \leq s + p \cdot (1 - s)$$

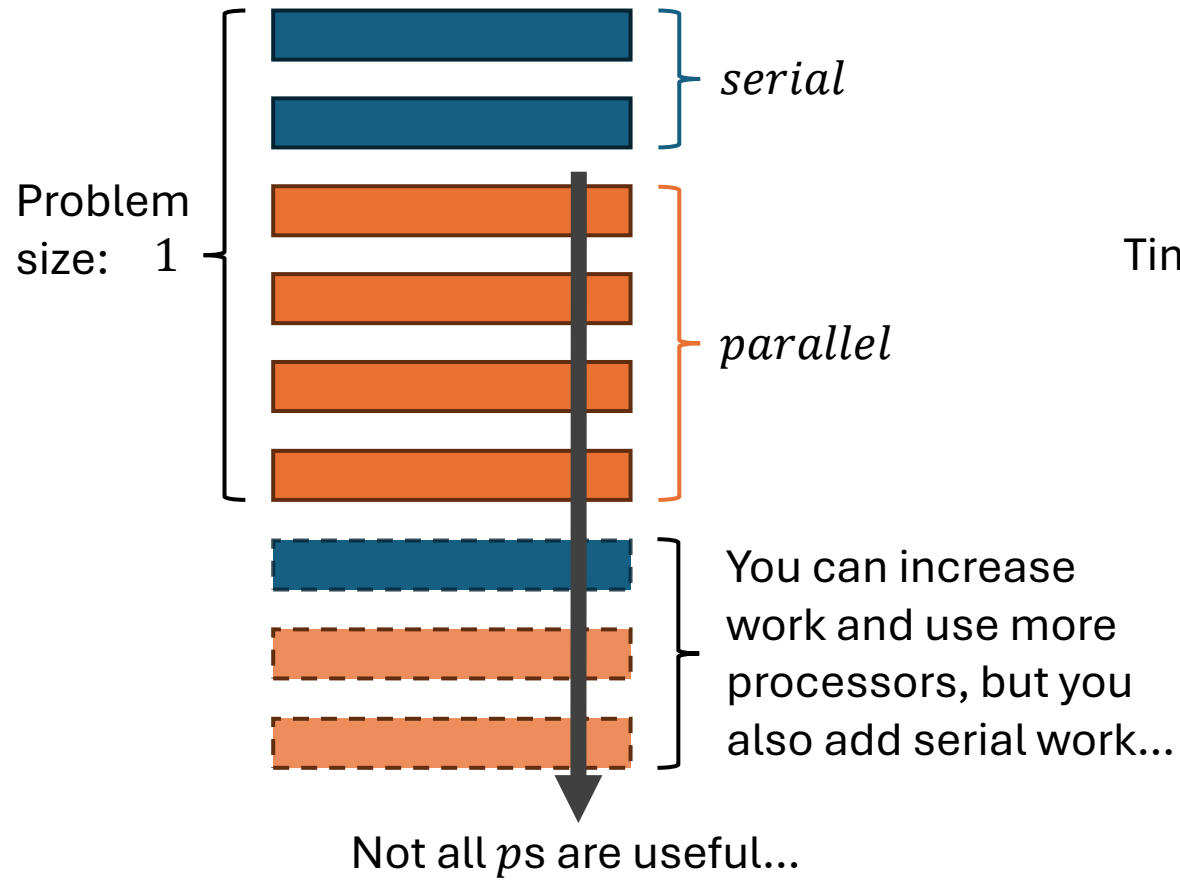
This is our fixed time.

This is = if $T^* = T_1$

Goal: run a larger problem in same amount of time

In practice, there is some work $n \cdot s$ done serially before going in parallel, but then the parallel work can be n , $10 \cdot n$, $100 \cdot n$, ... and if $p = n$ we can always finish in the same amount of time.

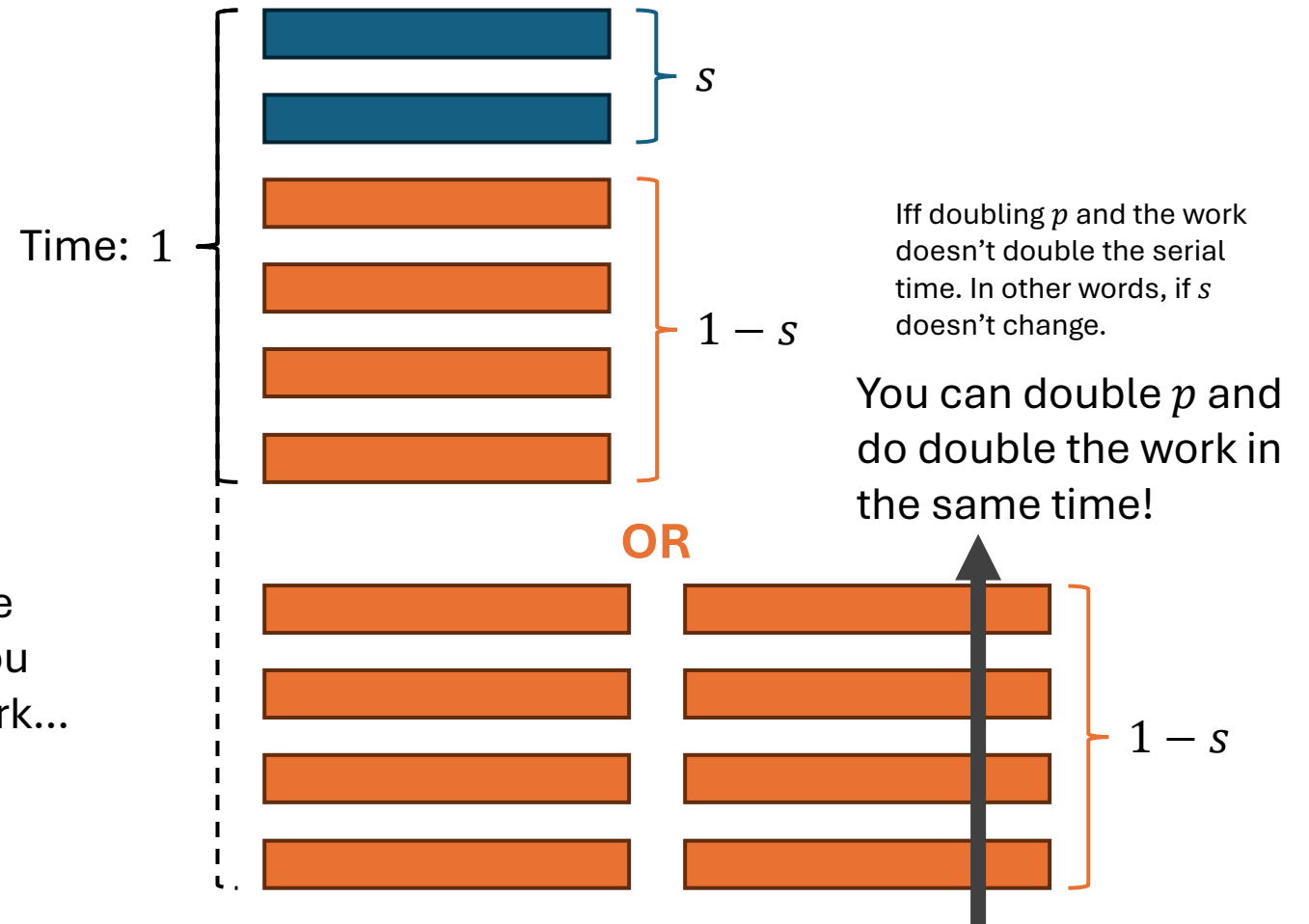
Amdahl's Law



SERIAL

PARALLEL

Gustafson's Law



Scaling Laws Exercise 1

A program loads a dataset of n tiny images from a file, this takes $10n$ cycles. Then proceeds to run a small CNN on each image (yes, on a CPU, deal with it). Each CNN inference takes 10^3 cycles.

- 1) What is the maximum possible speedup with an infinite number of processors?
- 2) What is the speedup with 4 processors? And with 64?
- 3) What is the cycles span (critical path) of the program with 16 processors?

Scaling Laws Exercise 1

- 1) Amount of work is fixed, number of processors varies: Amdahl! Thus, in cycles, $serial = 10n$, $parallel = 10^3n$

$$SU_{\infty} = \frac{serial + parallel}{serial} = \frac{1010}{10} = 101$$

- 2) With p processors:

$$SU_p = \frac{1010}{\frac{1000}{p} + 10} \Rightarrow SU_4 = \frac{1010}{260}, \quad SU_{64} = \frac{1010}{25,625}$$

- 3) With $p = 16$, some unlucky processors may do more work, taking $10^3 \cdot \left\lceil \frac{n}{16} \right\rceil$ cycles, so the span is $10n + 10^3 \cdot \left\lceil \frac{n}{16} \right\rceil$.

Scaling Laws Exercise 2

Ok, let's go with a simpler problem. A workload consists of 5% sequential operations and 95% parallelizable operations.

Assume $T^* = T_1$.

- 1) Compute the scaled speedup with 32 processors.
- 2) How does this compare to the speedup predicted by Amdahl's Law for the same parameters?

Scaling Laws Exercise 2

1) This is Gustafson: $SU_{32} = 0.05 + 32 \cdot 0.95 \approx 30.5$

2) For Amdahl: $SU_p = \frac{1}{\text{serial} + \frac{\text{parallel}}{p}}$, thus $SU_p = \frac{1}{0.05 + \frac{0.95}{32}} \approx 12.6$

TL;DR: Amdahl's law is pessimistic when problem size is fixed, while Gustafson's law shows the potential benefit if the workload grows with processor count.

Scaling Laws Exercise 3

Both scaling laws **assume ideal parallelism**. How would communication/synchronization overheads modify the predictions? Can you suggest a modified formula or rule-of-thumb?

Scaling Laws Exercise 3

Both scaling laws **assume ideal parallelism**. How would communication/synchronization overheads modify the predictions? Can you suggest a modified formula or rule-of-thumb?

Idea: add an overhead term $O(p)$ to T_p , representing the extra time per job due to parallelization management.

This way T_1 is unaffected, as it should be.

E.g., in Amdahl we would have $T_p = \text{serial} + \frac{\text{parallel}}{p} + O(p)$, while in Gustafson that would be $T_p = 1 + O(p)$.

Scaling Laws Exercise 4

Provide a typical example of an application where Amdahl's law is more representative than Gustafson. Then viceversa.

Scaling Laws Exercise 4

Provide a typical example of an application where Amdahl's law is more representative than Gustafson. Then viceversa.

Possible answers:

- 1) Amdahl: interactive and real-time system, with fixed small inputs and strict latency requirements. Simply, serial overhead matters more when the task is “a burst”. E.g. on-the-fly audio processing.
- 2) Gustafson: scientific simulations, where you increase resolution or model complexity along with cores (e.g. climate models, CFD).

Scaling Laws Exercise 5

If you had infinitely many processors, Amdahl's law says speedup is limited by the serial fraction. Gustafson's law says speedup can scale linearly. Who is “right”? Under what assumptions?

Scaling Laws Exercise 5

If you had infinitely many processors, Amdahl's law says speedup is limited by the serial fraction. Gustafson's law says speedup can scale linearly. Who is "right"? Under what assumptions?

Both are right!

Assumption: total work (problem size) is fixed.

In Amdahl the serial fraction dominates because the parallel part is of fixed size and gets quickly reduced to nothing by many processors.

With a non-zero serial fraction, the speedup saturates.

Scaling Laws Exercise 5

If you had infinitely many processors, Amdahl's law says speedup is limited by the serial fraction. Gustafson's law says speedup can scale linearly. Who is “right”? Under what assumptions?

Both are right!

Assumption: work grows with the number of processors.

The serial part is paid once for an arbitrarily large amount of work spread evenly among a proportional number of processors.

The speedup grows ~linearly with p , the parallel portion dominates.

Scaling Laws Exercise 5

TL;DR:

- If your program must handle the same small data set faster, Amdahl is the right model. The serial parts cap scaling.
- If you can do more work when more processors are available, Gustafson is the right model. You can keep all processors busy by increasing the parallel work.
- It's up to you to pick the scaling law that best models your scenario!